

Creating Employee Objects and Iterating with `forEach()`

Here's a complete demonstration of creating Employee objects and using `forEach()` with different iteration styles:

1. Employee Class Definition

java

```
import java.util.*;
import java.util.function.Consumer;

class Employee {
    private int id;
    private String name;
    private String department;
    private double salary;

    // Constructor
    public Employee(int id, String name, String department, double salary) {
        this.id = id;
        this.name = name;
        this.department = department;
        this.salary = salary;
    }

    // Getters
    public int getId() { return id; }
    public String getName() { return name; }
    public String getDepartment() { return department; }
    public double getSalary() { return salary; }

    // Setters
    public void setId(int id) { this.id = id; }
    public void setName(String name) { this.name = name; }
    public void setDepartment(String department) { this.department = department; }
    public void setSalary(double salary) { this.salary = salary; }

    @Override
```

```
public String toString() {
    return String.format("Employee{id=%d, name='%s', dept='%s', salary=%.2f}",
        id, name, department, salary);
}
}
```

2. Creating List of Employees

java

```
public class EmployeeForEachDemo {
    public static void main(String[] args) {

        // Method 1: Create using Arrays.asList()
        List<Employee> employees1 = Arrays.asList(
            new Employee(101, "John Doe", "IT", 75000),
            new Employee(102, "Jane Smith", "HR", 68000),
            new Employee(103, "Bob Johnson", "IT", 82000),
            new Employee(104, "Alice Brown", "Finance", 91000),
            new Employee(105, "Charlie Wilson", "Marketing", 62000)
        );

        // Method 2: Create using ArrayList and add()
        List<Employee> employees = new ArrayList<>();
        employees.add(new Employee(101, "John Doe", "IT", 75000));
        employees.add(new Employee(102, "Jane Smith", "HR", 68000));
        employees.add(new Employee(103, "Bob Johnson", "IT", 82000));
        employees.add(new Employee(104, "Alice Brown", "Finance", 91000));
        employees.add(new Employee(105, "Charlie Wilson", "Marketing", 62000));
        employees.add(new Employee(106, "David Lee", "IT", 79000));
        employees.add(new Employee(107, "Emma Davis", "HR", 72000));
        employees.add(new Employee(108, "Frank Miller", "Finance", 88000));

        System.out.println("=== Total Employees: " + employees.size() + "\n");

        // =====
        // DIFFERENT WAYS TO USE forEach()
        // =====

        // 1. Using Lambda Expression (Most Common)
```

```

System.out.println("--- 1. Lambda Expression ---");
employees.forEach(emp -> System.out.println(emp));

// 2. Using Method Reference
System.out.println("\n--- 2. Method Reference ---");
employees.forEach(System.out::println);

// 3. Using Lambda with Multiple Statements (Block Lambda)
System.out.println("\n--- 3. Block Lambda ---");
employees.forEach(emp -> {
    System.out.println("ID: " + emp.getId() + ", Name: " + emp.getName() +
        ", Dept: " + emp.getDepartment() + ", Salary: $" +
emp.getSalary());
});

// 4. Using Consumer Interface (Traditional way)
System.out.println("\n--- 4. Consumer Interface ---");
Consumer<Employee> consumer = new Consumer<Employee>() {
    @Override
    public void accept(Employee emp) {
        System.out.println(emp.getName() + " (" + emp.getDepartment() + ") - $" +
emp.getSalary());
    }
};
employees.forEach(consumer);

// 5. Using Custom Consumer (Lambda stored in variable)
System.out.println("\n--- 5. Custom Consumer Lambda ---");
Consumer<Employee> employeePrinter = emp ->
    System.out.printf("ID: %d | %-15s | %-10s | $%,.2f%n",
        emp.getId(), emp.getName(), emp.getDepartment(),
emp.getSalary());
employees.forEach(employeePrinter);
}
}

```

3. Advanced forEach() Operations

```
import java.util.*;
import java.util.stream.Collectors;

public class AdvancedForEachDemo {
    public static void main(String[] args) {

        List<Employee> employees = Arrays.asList(
            new Employee(101, "John Doe", "IT", 75000),
            new Employee(102, "Jane Smith", "HR", 68000),
            new Employee(103, "Bob Johnson", "IT", 82000),
            new Employee(104, "Alice Brown", "Finance", 91000),
            new Employee(105, "Charlie Wilson", "Marketing", 62000),
            new Employee(106, "David Lee", "IT", 79000),
            new Employee(107, "Emma Davis", "HR", 72000),
            new Employee(108, "Frank Miller", "Finance", 88000),
            new Employee(109, "Grace Taylor", "Marketing", 65000),
            new Employee(110, "Henry Adams", "IT", 85000)
        );

        // 1. Conditional Printing (IT Department only)
        System.out.println("=== IT Department Employees ===");
        employees.forEach(emp -> {
            if (emp.getDepartment().equals("IT")) {
                System.out.println(emp.getName() + " - $" + emp.getSalary());
            }
        });

        // 2. Sum of Salaries
        System.out.println("\n=== Salary Calculation ===");
        final double[] totalSalary = {0};
        employees.forEach(emp -> totalSalary[0] += emp.getSalary());
        System.out.printf("Total Salary: $%,.2f%n", totalSalary[0]);
        System.out.printf("Average Salary: $%,.2f%n", totalSalary[0] / employees.size());

        // 3. Department-wise Employee Count
        System.out.println("\n=== Department-wise Count ===");
        Map<String, Integer> deptCount = new HashMap<>();
        employees.forEach(emp -> {
            deptCount.put(emp.getDepartment(),
                deptCount.getDefault(emp.getDepartment(), 0) + 1);
        });
    }
}
```

```

deptCount.forEach((dept, count) ->
    System.out.println(dept + ": " + count + " employees"));

// 4. Update Salary (10% hike for all)
System.out.println("\n=== After 10% Salary Hike ===");
employees.forEach(emp -> emp.setSalary(emp.getSalary() * 1.10));
employees.forEach(emp ->
    System.out.printf("%-15s: $%,.2f%n", emp.getName(), emp.getSalary()));

// 5. Print employees with salary > 80,000
System.out.println("\n=== High Salary Employees (> $80,000) ===");
employees.forEach(emp -> {
    if (emp.getSalary() > 80000) {
        System.out.println(emp.getName() + " - $" + emp.getSalary());
    }
});
}
}

```

4. Using forEach() with Stream API for Better Performance

java

```

import java.util.*;
import java.util.stream.Stream;

public class StreamForEachDemo {
    public static void main(String[] args) {

        List<Employee> employees = Arrays.asList(
            new Employee(101, "John Doe", "IT", 75000),
            new Employee(102, "Jane Smith", "HR", 68000),
            new Employee(103, "Bob Johnson", "IT", 82000),
            new Employee(104, "Alice Brown", "Finance", 91000),
            new Employee(105, "Charlie Wilson", "Marketing", 62000)
        );

        // 1. Sequential Stream
        System.out.println("=== Sequential Stream ===");
        employees.stream().forEach(emp -> System.out.println(emp.getName()));
    }
}

```

```

// 2. Parallel Stream (for large datasets)
System.out.println("\n=== Parallel Stream (Order not guaranteed) ===");
employees.parallelStream().forEach(emp ->
    System.out.println(Thread.currentThread().getName() + " - " + emp.getName()));

// 3. forEachOrdered() - maintains order even in parallel stream
System.out.println("\n=== forEachOrdered() maintains order ===");
employees.parallelStream().forEachOrdered(emp ->
    System.out.println(emp.getId() + ": " + emp.getName()));

// 4. Chaining operations before forEach
System.out.println("\n=== Filtered and Sorted ===");
employees.stream()
    .filter(emp -> emp.getSalary() > 70000)
    .sorted(Comparator.comparing(Employee::getName))
    .forEach(emp -> System.out.println(emp.getName() + " - $" + emp.getSalary()));

// 5. Print specific fields
System.out.println("\n=== Employee Names Only ===");
employees.stream()
    .map(Employee::getName)
    .forEach(System.out::println);
}
}

```

5. Complete Output Example

text

```

=== Total Employees: 8

--- 1. Lambda Expression ---
Employee{id=101, name='John Doe', dept='IT', salary=75000.00}
Employee{id=102, name='Jane Smith', dept='HR', salary=68000.00}
Employee{id=103, name='Bob Johnson', dept='IT', salary=82000.00}
Employee{id=104, name='Alice Brown', dept='Finance', salary=91000.00}
Employee{id=105, name='Charlie Wilson', dept='Marketing', salary=62000.00}
Employee{id=106, name='David Lee', dept='IT', salary=79000.00}
Employee{id=107, name='Emma Davis', dept='HR', salary=72000.00}

```

```
Employee{id=108, name='Frank Miller', dept='Finance', salary=88000.00}
```

```
--- 2. Method Reference ---
```

```
Employee{id=101, name='John Doe', dept='IT', salary=75000.00}
```

```
Employee{id=102, name='Jane Smith', dept='HR', salary=68000.00}
```

```
... (continues)
```

```
=== IT Department Employees ===
```

```
John Doe - $75000.0
```

```
Bob Johnson - $82000.0
```

```
David Lee - $79000.0
```

```
Henry Adams - $85000.0
```

```
=== Salary Calculation ===
```

```
Total Salary: $765,000.00
```

```
Average Salary: $76,500.00
```

```
=== Department-wise Count ===
```

```
IT: 4 employees
```

```
HR: 2 employees
```

```
Finance: 2 employees
```

```
Marketing: 2 employees
```

6. Important Notes about forEach()

Feature	Description
Order	For List, iteration follows insertion order
Performance	Parallel streams can improve performance
Side Effects	Should not modify collection while iterating (ConcurrentModificationException)
Lambdas	Variables used must be effectively final
Stream vs Collection	Stream.forEach() and Collection.forEach() work similarly

Best Practices:

1. Use method references (`::`) for simple operations
2. Use lambda expressions for complex operations
3. Avoid modifying the collection during iteration
4. Use parallel streams only for large datasets
5. Prefer `forEachOrdered()` when order matters